# 🚀 AWS Messaging & Event-Driven — Category 9: Complete Interview Guide

> **Target Audience:** DevOps / SRE / Platform / Cloud Engineers
> **Difficulty:** Beginner → Advanced
> **Topics Covered:** SQS Standard vs FIFO, Visibility Timeout, DLQ, SNS Fan-out, Long Polling, EventBridge, Kinesis Streams, Kinesis Firehose, MSK, Exactly-Once Processing, Poison Pill Handling

---

## 📋 Table of Contents


1. 9.1 SQS — Standard vs FIFO, Visibility Timeout, DLQ
2. 9.2 SNS — Topics, Subscriptions, Fan-out Pattern
3. 9.3 SQS — Long Polling, Message Attributes, Delay Queues
4. 9.4 EventBridge — Event Buses, Rules, Patterns, Pipes
5. 9.5 SNS + SQS Fan-out Architecture
6. 9.6 Kinesis Data Streams — Shards, Sequence Numbers, Enhanced Fan-out
7. 9.7 Kinesis Firehose — Delivery Streams, Transformation, Buffering
8. 9.8 MSK — Managed Kafka vs Kinesis, Use Cases
9. 9.9 SQS — Exactly-Once Processing Patterns, Poison Pill Handling


---

# 9.1 SQS — Standard vs FIFO, Visibility Timeout, DLQ

---

## 🟢 What It Is in Simple Terms

SQS (Simple Queue Service) is AWS's fully managed message queue. It decouples producers (systems that create work) from consumers (systems that do the work). The producer sends a message to the queue and moves on. The consumer pulls messages from the queue at its own pace. Neither needs to know the other exists.

---

## 🔍 Why It Exists / What Problem It Solves

```
Without SQS — tight coupling:
User clicks "upload photo"
→ App calls resize service directly (synchronous HTTP)
→ If resize service is slow: user waits
→ If resize service is down: request fails
→ If traffic spikes: resize service is overwhelmed

With SQS — loose coupling:
User clicks "upload photo"
→ App sends message to SQS queue (instant, always available)
```

```
→ User gets immediate response: "processing"
→ Resize workers pull from queue at their own pace
→ Resize service down? Messages wait in queue (up to 14 days)
→ Traffic spike? Queue absorbs burst, workers scale gradually
```

---

## ⚙️ How SQS Works Internally

```
SQS Message Lifecycle:

Producer sends message
        │
        ▼
┌──────────────────────────────────────────────────────────┐
│                      SQS Queue                           │
│  [msg1] [msg2] [msg3] [msg4] [msg5]                      │
└──────────────────────────────────────────────────────────┘
        │
        ▼ Consumer polls (ReceiveMessage API call)
Consumer receives msg1 → msg1 becomes INVISIBLE (visibility timeout
starts)
        │
        ├── Processing succeeds → consumer calls DeleteMessage → gone
        └── Processing fails / timeout expires → msg1 becomes VISIBLE
again
                                                  → another consumer can
pick it up

This at-least-once delivery model means:
→ A message CAN be delivered more than once (if timeout expires before
delete)
→ Consumers MUST be idempotent (processing same message twice = same
result)
```

---

## 🧩 Standard Queue vs FIFO Queue

```
┌──────────────────────┬──────────────────────┬──────────────────────┐
│ Feature              │ Standard Queue       │ FIFO Queue           │
├──────────────────────┼──────────────────────┼──────────────────────┤
│ Ordering             │ Best-effort          │ Strict FIFO          │
│                      │ (usually in order,   │ (guaranteed within   │
│                      │  but not guaranteed) │  message group)      │
│ Delivery             │ At-least-once        │ Exactly-once         │
│                      │ (duplicates possible)│ (deduplication)      │
│ Throughput           │ Unlimited TPS        │ 300 TPS (no batching)│
│                      │                      │ 3,000 TPS (batching) │
│ Use cases            │ Most workloads       │ Financial txns,      │
│                      │ Email notifications  │ order processing,    │
```

```
│                        │ Log processing        │ inventory updates    │
│ Queue name suffix      │ None                  │ Must end in .fifo    │
│ MessageGroupId         │ Not used              │ Required             │
│ MessageDeduplicationId│ Not used              │ Required or content  │
│                        │                       │ deduplication        │
└────────────────────────┴───────────────────────┴──────────────────────┘

FIFO MessageGroupId:
├── All messages with same GroupId processed in strict FIFO order
├── Different GroupIds processed in parallel (concurrent processing)
└── Example: GroupId = orderId → all order events ordered per order
             GroupId = customerId → all customer events ordered per
customer

FIFO deduplication:
├── MessageDeduplicationId: explicit unique ID per message
└── Content-based deduplication: SHA-256 hash of body
    If same hash arrives within 5-minute deduplication window → silently
dropped
```

```aws
# Create standard queue
aws sqs create-queue \
  --queue-name prod-orders \
  --attributes '{
    "VisibilityTimeout": "300",
    "MessageRetentionPeriod": "86400",
    "ReceiveMessageWaitTimeSeconds": "20"
  }'

# Create FIFO queue
aws sqs create-queue \
  --queue-name prod-payments.fifo \
  --attributes '{
    "FifoQueue": "true",
    "ContentBasedDeduplication": "true",
    "VisibilityTimeout": "300"
  }'

# Send message to FIFO queue
aws sqs send-message \
  --queue-url https://sqs.us-east-1.amazonaws.com/123/prod-payments.fifo \
  --message-body '{"amount": 99.99, "currency": "USD"}' \
  --message-group-id "order-456" \
  --message-deduplication-id "txn-789"
```

---

## 🧩 Visibility Timeout

```
Visibility Timeout = how long a message is hidden from other consumers
                     after one consumer receives it

Default: 30 seconds
Range:   0 seconds to 12 hours (43,200 seconds)

The fundamental rule:
Visibility Timeout MUST be longer than your maximum processing time.

If processing takes 5 minutes and timeout is 30 seconds:
→ Consumer receives message, starts processing
→ 30 seconds pass → timeout expires → message becomes visible again
→ Second consumer receives SAME message → duplicate processing!
→ First consumer finishes at 5 minutes → tries DeleteMessage
→ Success (but damage already done — message was processed twice)

Visibility Timeout use cases:
├── Short tasks (image thumbnail):     30-60 seconds
├── Medium tasks (data processing):    5-15 minutes
├── Long tasks (video encoding):       1-12 hours (or use
ChangeVisibilityTimeout)
└── Lambda consumers: MUST set timeout to 6× Lambda function timeout
The 6× multiplier exists because SQS checks visibility on a polling
interval and needs enough headroom to avoid redelivery before Lambda
finishes, retries, and calls DeleteMessage. Example: Lambda timeout = 10s
→ set visibility timeout = 60s minimum.

Extending visibility timeout during processing:
# Consumer calls ChangeMessageVisibility to extend the timeout
aws sqs change-message-visibility \
  --queue-url https://sqs.us-east-1.amazonaws.com/123/prod-orders \
  --receipt-handle "AQEBxyz..." \
  --visibility-timeout 600  # extend to 10 more minutes

# Pattern: heartbeat while processing
# Every 4 minutes, extend by 5 minutes → never expires during long
processing
```

---

## 🧩 Dead Letter Queue (DLQ)

```
DLQ = a separate queue for messages that repeatedly fail processing

When to use DLQ:
├── Consumer fails to process a message after N attempts
├── Message is "poisoned" (malformed data, impossible to process)
├── Without DLQ: message retried forever → consumers stuck
└── With DLQ:    after maxReceiveCount retries → moved to DLQ

DLQ flow:
```

```
Queue: msg1 → consumer picks up → processing fails → msg1 visible again
       msg1 → consumer picks up → processing fails → msg1 visible again
       msg1 → consumer picks up → processing fails → receiveCount =
maxReceiveCount
       msg1 → automatically moved to Dead Letter Queue

┌──────────────────────┐    after maxReceiveCount    ┌──────────────────┐
│  Source Queue        │ ──────────────────────────► │  Dead Letter     │
│  prod-orders         │    failures exceeded        │  Queue           │
│  maxReceiveCount: 3  │                             │  prod-orders-dlq │
└──────────────────────┘                             └──────────────────┘
                                                              │
                                                              ▼
                                                     Alert ops team
                                                     Investigate failure
                                                     Fix and replay

Setting up DLQ:
# Step 1: Create DLQ
aws sqs create-queue --queue-name prod-orders-dlq

# Step 2: Get DLQ ARN
DLQ_ARN=$(aws sqs get-queue-attributes \
  --queue-url https://sqs.../prod-orders-dlq \
  --attribute-names QueueArn \
  --query 'Attributes.QueueArn' --output text)

# Step 3: Configure source queue with redrive policy
aws sqs set-queue-attributes \
  --queue-url https://sqs.../prod-orders \
  --attributes '{
    "RedrivePolicy": "
{\"deadLetterTargetArn\":\"'"$DLQ_ARN"'\",\"maxReceiveCount\":\"3\"}"
  }'

DLQ message retention:
├── DLQ should have LONGER retention than source queue
│   Source: 4 days, DLQ: 14 days (gives time to investigate)
└── DLQ itself can have a DLQ (for DLQ processing failures)

DLQ redrive (replay):
After fixing the bug, replay DLQ messages back to source queue:
aws sqs start-message-move-task \
  --source-arn arn:aws:sqs:...:prod-orders-dlq \
  --destination-arn arn:aws:sqs:...:prod-orders

Key metrics to alarm on:
├── ApproximateNumberOfMessagesNotVisible: messages currently being
processed
├── ApproximateNumberOfMessagesVisible:   messages waiting to be processed
└── NumberOfMessagesSent to DLQ:          failing messages (alarm on any >
0)
```

---

## 💬 Short Crisp Interview Answer

*"SQS decouples producers from consumers through an asynchronous message queue. Standard queues offer unlimited throughput with at-least-once, best-effort-ordering delivery — duplicates are possible so consumers must be idempotent. FIFO queues guarantee strict ordering within message groups and exactly-once delivery using deduplication IDs, but are limited to 3,000 TPS with batching. Visibility timeout hides a message from other consumers while one consumer processes it — it must be longer than your maximum processing time or you get duplicate processing. The DLQ catches messages that fail repeatedly: after maxReceiveCount failures, the message moves to the DLQ for investigation, preventing poison messages from blocking healthy processing indefinitely."*

---

## ⚠ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| Visibility timeout too short | Processing takes longer than timeout → duplicate delivery → must handle idempotency |
| Lambda SQS timeout rule | Lambda visibility timeout MUST be at least 6× the Lambda function timeout |
| Standard queue delivers duplicates | Always design consumers to be idempotent — duplicates WILL happen |
| DLQ must be same type | Standard queue DLQ must be Standard. FIFO queue DLQ must be FIFO |
| Message retention default | Default 4 days retention. Set to 14 days for DLQ to give investigation time |
| DeleteMessage is required | Consuming a message does NOT delete it — you must explicitly call DeleteMessage |

---

# 9.2 SNS — Topics, Subscriptions, Fan-out Pattern

---

## 🟢 What It Is in Simple Terms

SNS (Simple Notification Service) is AWS's publish-subscribe messaging service. A producer publishes one message to an SNS topic, and SNS simultaneously delivers that message to all subscribers — whether that's 1 subscriber or 10 million. It's push-based: SNS pushes to subscribers, unlike SQS where consumers pull.

---

## 🔍 Why It Exists / What Problem It Solves

```
Without SNS:
Order service creates order → must notify:
  - Email service
  - Inventory service
  - Fraud detection
  - Analytics service
  - Shipping service

Tight coupling: Order service makes 5 separate HTTP calls.
One service down = order service fails or needs complex retry logic.

With SNS:
Order service publishes one message to "orders" topic.
SNS delivers simultaneously to all 5 subscribers.
Order service has zero knowledge of downstream consumers.
Adding a 6th consumer = subscribe to topic, zero order service changes.
```

---

## 🧩 SNS Core Components

```
SNS Architecture:

Publisher                    SNS Topic                    Subscribers
                            ┌──────────────┐
                            │  orders-topic │
Order Service ──Publish──►│              │──► SQS Queue (email-svc)
                            │              │──► SQS Queue (inventory-svc)
                            │              │──► Lambda (fraud-detection)
                            │              │──► HTTP/HTTPS endpoint
(analytics)
                            │              │──► Email (ops@company.com)
                            │              │──► SMS (+1-555-0123)
                            └──────────────┘

SNS supported subscription protocols:
├── SQS:        deliver to SQS queue (most common — adds buffering)
├── Lambda:     invoke Lambda function directly
├── HTTP/HTTPS: POST to an HTTP endpoint
├── Email:      send email (human-readable, not for machine processing)
├── Email-JSON: send raw JSON to email
├── SMS:        send text message to phone number
├── Mobile Push: APNS (iOS), FCM (Android), ADM (Amazon)
└── Kinesis Firehose: stream to delivery stream

SNS message structure:
{
  "Type": "Notification",
  "MessageId": "abc-123-def",
  "TopicArn": "arn:aws:sns:us-east-1:123:orders-topic",
  "Subject": "New Order Created",
```

```
  "Message": "{\"orderId\":\"ord-456\",\"amount\":99.99}",
  "Timestamp": "2024-01-15T14:30:00.000Z",
  "SignatureVersion": "1",
  "Signature": "...",     ← HTTPS endpoint verification
  "UnsubscribeURL": "..."
}
```

---

## 🧩 SNS Topic Types

```
Standard Topic:
├── Best-effort ordering (usually delivered in order, not guaranteed)
├── At-least-once delivery (duplicates possible)
├── Unlimited throughput
└── Supports all subscription protocols

FIFO Topic:
├── Strict message ordering
├── Exactly-once delivery
├── Limited to: SQS FIFO queues as subscribers ONLY
│   (no Lambda, HTTP, email, SMS for FIFO topics)
├── Up to 300 published messages/second (3,000 with batching)
└── Use for: financial events, inventory updates where order matters
```

---

## 🧩 Message Filtering

```
Message filtering = subscribers receive ONLY the messages they care about
                    Publisher sends one message with attributes
                    Each subscription has a filter policy
                    SNS evaluates filter → delivers only matching messages

Without filtering: ALL subscribers receive ALL messages
With filtering:    each subscriber only receives relevant messages

Example: Order events with different event types

Publisher sends message with attributes:
{
  "MessageAttributes": {
    "eventType": {"DataType": "String", "StringValue": "ORDER_SHIPPED"},
    "region":    {"DataType": "String", "StringValue": "us-east-1"},
    "amount":    {"DataType": "Number", "StringValue": "99.99"}
  },
  "Message": "{\"orderId\":\"ord-456\"}"
}

Subscription filter policies:
Email ops team — receives ALL events:
```

```
  {} (empty policy = no filter = receives everything)

Shipping service SQS — receives ONLY shipping events:
  {"eventType": ["ORDER_SHIPPED", "ORDER_DELIVERED"]}

Fraud detection Lambda — receives high-value orders:
  {"amount": [{"numeric": [">=", 1000]}]}

High-value US orders:
  {"region": ["us-east-1", "us-west-2"], "amount": [{"numeric": [">",
500]}]}

Filter policy scope options:
├── MessageAttributes (default): filter on message attributes
└── MessageBody: filter on JSON fields inside the message body itself
```

```
# Create SNS topic
aws sns create-topic --name prod-orders

# Subscribe SQS queue with filter policy
aws sns subscribe \
  --topic-arn arn:aws:sns:us-east-1:123:prod-orders \
  --protocol sqs \
  --notification-endpoint arn:aws:sqs:us-east-1:123:shipping-queue \
  --attributes '{
    "FilterPolicy": "{\"eventType\": [\"ORDER_SHIPPED\",
\"ORDER_DELIVERED\"]}",
    "FilterPolicyScope": "MessageAttributes"
  }'

# Publish message with attributes
aws sns publish \
  --topic-arn arn:aws:sns:us-east-1:123:prod-orders \
  --message '{"orderId":"ord-456","amount":99.99}' \
  --message-attributes '{
    "eventType": {"DataType":"String","StringValue":"ORDER_SHIPPED"},
    "amount":    {"DataType":"Number","StringValue":"99.99"}
  }'
```

---

## 🧩 SNS Delivery Guarantees and Retry

```
SNS delivery behavior:
├── At-least-once delivery (not exactly-once)
├── Synchronous delivery attempt first
└── If delivery fails → retry with exponential backoff

HTTP/HTTPS subscriber retry policy:
├── Immediate retries: 3 attempts (no delay)
```

```
  ├── Pre-backoff phase: 2 attempts (1-second delay)
  ├── Backoff phase: 10 attempts (exponential, max 20 seconds)
  └── Post-backoff phase: 100,000 attempts (20-second intervals)
      Total: delivery attempted for up to ~23 days!

  SQS subscriber retries:
  ├── SNS retries SQS delivery for up to 23 days
  └── SQS itself provides its own DLQ for failed processing

  Dead Letter Queue for SNS:
  ├── If ALL delivery retries exhausted → message goes to SNS DLQ
  ├── SNS DLQ captures undeliverable messages
  └── Set DLQ on each subscription (not the topic itself)

  aws sns set-subscription-attributes \
    --subscription-arn arn:aws:sns:...:subscription-id \
    --attribute-name RedrivePolicy \
    --attribute-value '{"deadLetterTargetArn":"arn:aws:sqs:...:sns-dlq"}'
```

---

## 💬 Short Crisp Interview Answer

*"SNS is a pub-sub service where one publisher sends to a topic and SNS pushes to all subscribers simultaneously — unlike SQS where consumers pull. Subscribers can be SQS queues, Lambda functions, HTTP endpoints, email, SMS, or mobile push. Message filtering lets each subscriber define a filter policy so it only receives relevant messages — shipping subscribes with eventType=ORDER_SHIPPED, fraud detection subscribes with amount >= 1000 — one topic serves all consumers cleanly. SNS delivers at-least-once, so SQS subscribers should use their DLQ for undeliverable messages. The most important architectural pattern is SNS+SQS fan-out: instead of subscribing Lambda or HTTP directly to SNS, subscribe SQS queues — this adds buffering, retry, and DLQ to every subscriber independently."*

---

## ⚠ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| SNS is push-based | SNS pushes to subscribers — consumers cannot control receive rate like SQS pull |
| FIFO topic = SQS FIFO only | FIFO topics cannot deliver to Lambda, HTTP, Email, or SMS |
| Message size limit | SNS max message size: 256KB. Large payloads need S3 reference pattern |
| Filter policy = JSON | Must be valid JSON. Empty filter {} = receive all messages |
| HTTP subscriber confirmation | HTTP/HTTPS endpoints must confirm subscription by responding to SubscriptionConfirmation message |
| SNS doesn't persist messages | SNS is not a queue — if subscriber is unavailable and retries exhausted, message is lost (use SQS) |

---

# 9.3 SQS — Long Polling, Message Attributes, Delay Queues

## 🟢 What It Is in Simple Terms

Three operational SQS features that tune how and when messages are received. Long polling reduces cost by waiting for messages instead of returning empty responses. Message attributes attach metadata to messages for filtering or routing. Delay queues postpone message visibility for a configurable time after sending.

## 🧩 Long Polling vs Short Polling

```
Short Polling (default — ReceiveMessageWaitTimeSeconds=0):
├── Consumer calls ReceiveMessage → SQS responds immediately
│   (even if zero messages available — empty response)
├── High-volume empty responses → wasted API calls → cost
└── Example: 100 empty calls/minute = 144,000 empty calls/day = wasted
cost

Long Polling (ReceiveMessageWaitTimeSeconds=1-20):
├── Consumer calls ReceiveMessage → SQS WAITS up to 20 seconds
│   for at least 1 message to arrive before responding
├── If message arrives in 3 seconds → returns immediately with message
├── If no message in 20 seconds → returns empty (but only 1 API call!)
└── Dramatically reduces empty responses and API costs

Setting long polling:
Option 1: Queue-level (applies to all consumers of this queue)
aws sqs set-queue-attributes \
  --queue-url https://sqs.../prod-orders \
  --attributes '{"ReceiveMessageWaitTimeSeconds": "20"}'

Option 2: Per-call (override at receive time)
aws sqs receive-message \
  --queue-url https://sqs.../prod-orders \
  --wait-time-seconds 20 \
  --max-number-of-messages 10

Cost comparison:
Short polling: queue processes 10 messages/hour
→ Consumer polls every second → 3,600 API calls/hour
→ 3,590 empty calls → $0.036/day wasted

Long polling (20s):
→ Consumer polls every 20 seconds → 180 API calls/hour
→ 170 empty calls → $0.0017/day → 20x cheaper
```

```
⚠ Always use long polling in production.
   Set ReceiveMessageWaitTimeSeconds=20 on every queue.
   Short polling is acceptable only for tests.
```

---

## 🧩 Message Attributes

```
Message Attributes = metadata attached to a message
                     Sent alongside message body but separate from it
                     Consumers can inspect attributes without parsing body

Attribute data types:
├── String:  text values
├── Number:  integer or float values (stored as string internally)
├── Binary:  binary data (base64 encoded)
└── Custom:  String.MyType, Number.int, Binary.jpg

Attributes vs body:
├── Body: the actual message content (order JSON, event payload)
└── Attributes: routing metadata, message type, version, timestamps
    Consumers can make routing decisions from attributes
    without deserializing the full message body

Example: routing different event types
{
  "MessageBody": "{\"orderId\":\"ord-456\",\"items\":[...]}",
  "MessageAttributes": {
    "eventType": {
      "DataType": "String",
      "StringValue": "ORDER_CREATED"
    },
    "schemaVersion": {
      "DataType": "Number",
      "StringValue": "2"
    },
    "correlationId": {
      "DataType": "String",
      "StringValue": "req-abc-123"
    },
    "priority": {
      "DataType": "String",
      "StringValue": "HIGH"
    }
  }
}
```

```
# Send message with attributes
aws sqs send-message \
  --queue-url https://sqs.us-east-1.amazonaws.com/123/prod-orders \
```

```
  --message-body '{"orderId":"ord-456"}' \
  --message-attributes '{
    "eventType": {"DataType":"String","StringValue":"ORDER_CREATED"},
    "priority":  {"DataType":"String","StringValue":"HIGH"}
  }'

# Receive message and inspect attributes
aws sqs receive-message \
  --queue-url https://sqs.../prod-orders \
  --message-attribute-names All \
  --wait-time-seconds 20
```

```
Use cases for message attributes:
├── Routing:     lambda reads eventType, routes to appropriate handler
├── Filtering:   SNS filter policies read SQS message attributes
├── Tracing:     correlationId + traceId for distributed tracing
├── Versioning:  schemaVersion for backward-compatible schema evolution
└── Priority:    consumers can skip processing based on priority attribute
    (SQS itself does NOT prioritize — use separate queues for true
priority)

⚠ Max 10 message attributes per message.
   Attributes count toward the 256KB total message size limit.
```

---

## 🧩 Delay Queues

```
Delay Queue = postpone delivery of new messages for a fixed time period
              Messages are invisible to consumers until the delay expires

Default delay: 0 seconds (no delay)
Maximum delay: 15 minutes (900 seconds)

Two levels of delay:
├── Queue-level delay: applies to ALL messages sent to the queue
└── Per-message delay:  override delay for individual messages
    (available on Standard queues only — NOT on FIFO queues)

Setting queue-level delay:
aws sqs set-queue-attributes \
  --queue-url https://sqs.../prod-orders \
  --attributes '{"DelaySeconds": "60"}'  # 1 minute delay for all messages

Setting per-message delay:
aws sqs send-message \
  --queue-url https://sqs.../prod-orders \
  --message-body '{"orderId":"ord-456"}' \
  --delay-seconds 300  # this specific message delayed 5 minutes
```

```
Use cases for delay queues:
├── Retry after failure: send message with 30-second delay for delayed
retry
├── Scheduled processing: delay payment capture 24 hours after auth
├── Rate limiting: space out downstream API calls over time
├── Transactional delay: wait for DB transaction to commit before
processing
└── Email campaigns: delay welcome email 5 minutes after signup

Delay vs Visibility Timeout:
├── Delay:              hides message from consumers BEFORE first receipt
│                       applies at send time
└── Visibility Timeout: hides message from consumers DURING processing
                        applies after a consumer receives the message
```

---

## 💬 Short Crisp Interview Answer

*"Three important SQS operational features. Long polling sets ReceiveMessageWaitTimeSeconds up to 20 seconds — the call waits for messages instead of returning empty immediately, dramatically reducing wasted API calls and cost. Always set long polling to 20 on every production queue. Message attributes attach typed metadata alongside the message body — useful for routing decisions, schema versioning, and correlation IDs without parsing the full message payload. Delay queues postpone message visibility for up to 15 minutes after sending — useful for delayed retries, transactional delays, and rate limiting downstream systems. The key distinction from visibility timeout: delay applies before first receipt at send time, visibility timeout applies after a consumer receives the message during processing."*

---

## ⚠ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| Short polling is the default | ReceiveMessageWaitTimeSeconds defaults to 0. Always explicitly set to 20 |
| Per-message delay on FIFO | FIFO queues do NOT support per-message DelaySeconds — only queue-level delay |
| Max 10 attributes | SQS limits message attributes to 10 per message |
| Attribute size counts toward limit | Message attributes contribute to the 256KB total message size |
| Delay ≠ scheduler | Delay queues are not a precise scheduler — delays are approximate |

---

# 9.4 EventBridge — Event Buses, Rules, Patterns, Pipes

---

## 🟢 What It Is in Simple Terms

EventBridge is AWS's serverless event routing service. Events (JSON documents describing something that happened) are published to an event bus. Rules match events against patterns and route matching events to targets. It's the glue that connects AWS services and your applications through events without writing point-to-point integrations.

---

## 🔍 Why It Exists / What Problem It Solves

```
Without EventBridge:
EC2 instance terminates
→ Must poll EC2 API to detect termination
→ Update Route53, notify Slack, trigger Ansible playbook
→ 3 separate integrations, polling overhead, brittle code

With EventBridge:
EC2 instance terminates → automatically generates event
→ EventBridge rule matches {"source": "aws.ec2", "detail-type": "EC2
Instance State-change"}
→ Simultaneously routes to:
   - Lambda: deregister from Route53
   - SNS:    notify Slack
   - SSM:    trigger Ansible playbook
→ Zero polling, zero custom integration code, event-driven
```

---

## 🧩 Event Buses

```
EventBridge has three types of event buses:

1. Default event bus:
   ├── Receives events from AWS services automatically
   │   (EC2 state changes, RDS events, S3 notifications, GuardDuty
findings)
   ├── Every AWS account has exactly one default event bus
   └── Cannot delete or disable it

2. Custom event buses:
   ├── Created by you for your application events
   ├── Applications publish custom events to custom buses
   ├── Can receive events from other AWS accounts (cross-account)
   └── Create separate buses per domain/application for isolation

3. Partner event buses:
   ├── Events from AWS SaaS partners
   └── Examples: Datadog, PagerDuty, Zendesk, Shopify, GitHub
       Partner events arrive directly to your account without polling

Event bus cross-account:
Account A (producer) → publishes events to Account B event bus
```

```
Resource policy on Account B event bus allows Account A:
{
  "Effect": "Allow",
  "Principal": {"AWS": "arn:aws:iam::ACCOUNT-A:root"},
  "Action": "events:PutEvents",
  "Resource": "arn:aws:events:us-east-1:ACCOUNT-B:event-bus/custom-bus"
}
```

---

## 🧩 Event Structure

```
Every EventBridge event is a JSON document:
{
  "version": "0",
  "id": "abc-123-def",
  "source": "aws.ec2",                        ← who sent the event
  "account": "123456789012",
  "time": "2024-01-15T14:30:00Z",
  "region": "us-east-1",
  "resources": [
    "arn:aws:ec2:us-east-1:123:instance/i-0abc123"
  ],
  "detail-type": "EC2 Instance State-change Notification", ← event type
  "detail": {                                 ← event-specific data
    "instance-id": "i-0abc123",
    "state": "terminated"
  }
}

Custom event (from your application):
{
  "source": "myapp.orders",
  "detail-type": "OrderCreated",
  "detail": {
    "orderId": "ord-456",
    "customerId": "cust-789",
    "amount": 99.99,
    "currency": "USD"
  }
}
```

```
# Publish custom event
aws events put-events \
  --entries '[{
    "Source": "myapp.orders",
    "DetailType": "OrderCreated",
    "Detail": "{\"orderId\":\"ord-456\",\"amount\":99.99}",
    "EventBusName": "prod-orders-bus"
  }]'
```

---

## 🧩 Rules and Event Patterns

```
EventBridge Rule = pattern matcher + target router
                   When event matches pattern → send to target(s)

A single rule can route to up to 5 targets simultaneously.

Event pattern matching (subset matching — only specified fields checked):

Match any EC2 state change:
{"source": ["aws.ec2"]}

Match EC2 instance termination only:
{
  "source": ["aws.ec2"],
  "detail-type": ["EC2 Instance State-change Notification"],
  "detail": {
    "state": ["terminated"]
  }
}

Match GuardDuty HIGH severity findings:
{
  "source": ["aws.guardduty"],
  "detail-type": ["GuardDuty Finding"],
  "detail": {
    "severity": [{"numeric": [">=", 7]}]
  }
}

Match OrderCreated events over $1000:
{
  "source": ["myapp.orders"],
  "detail-type": ["OrderCreated"],
  "detail": {
    "amount": [{"numeric": [">", 1000]}]
  }
}

Pattern operators:
├── Exact match:    ["value1", "value2"]
├── Prefix:         [{"prefix": "order-"}]
├── Suffix:         [{"suffix": "-prod"}]
├── Numeric:        [{"numeric": [">=", 100]}]
├── Anything-but:   [{"anything-but": ["CANCELLED"]}]
├── Exists:         [{"exists": true}] or [{"exists": false}]
└── IP CIDR:        [{"cidr": "10.0.0.0/8"}]
```

```
# Create EventBridge rule
aws events put-rule \
  --name "high-value-orders" \
  --event-bus-name "prod-orders-bus" \
  --event-pattern '{
    "source": ["myapp.orders"],
    "detail-type": ["OrderCreated"],
    "detail": {"amount": [{"numeric": [">", 1000]}]}
  }' \
  --state ENABLED

# Add Lambda target to rule
aws events put-targets \
  --rule "high-value-orders" \
  --event-bus-name "prod-orders-bus" \
  --targets '[{
    "Id": "fraud-detection",
    "Arn": "arn:aws:lambda:...:function:fraud-detector",
    "InputTransformer": {
      "InputPathsMap": {
        "orderId": "$.detail.orderId",
        "amount":  "$.detail.amount"
      },
      "InputTemplate": "{\"order\": \"<orderId>\", \"value\": <amount>}"
    }
  }]'
```

---

## 🧩 EventBridge Targets

```
Supported targets (over 20 AWS services):
├── Lambda function        → invoke function with event as input
├── SQS queue              → send event as message body
├── SNS topic              → publish event as notification
├── Step Functions         → start execution with event as input
├── ECS task               → run task with event environment variables
├── Kinesis Data Streams   → put record to stream
├── Kinesis Firehose       → put record to delivery stream
├── API Gateway            → invoke HTTP endpoint
├── EventBridge bus        → forward to another bus (cross-account/region)
├── CloudWatch Logs        → log events directly
├── CodePipeline           → trigger pipeline execution
├── SSM RunCommand         → run commands on EC2 instances
└── API Destinations       → call ANY HTTP endpoint (third-party SaaS)

API Destinations:
├── Call ANY external HTTP/HTTPS endpoint from EventBridge
├── Example: create PagerDuty incident, post to Slack, trigger Jira
├── Supports: OAuth, API key, basic auth connection types
└── Rate limiting: configure max requests per second to destination
```

---

## 🧩 EventBridge Pipes

```
EventBridge Pipes = point-to-point integration between source and target
                    with optional filtering and enrichment in between

Pipes architecture:
Source → [Filter] → [Enrichment] → Target

Sources (polling-based):
├── SQS queue
├── DynamoDB Streams
├── Kinesis Data Streams
├── Kafka / MSK
└── RabbitMQ

Enrichment (optional transformation step):
├── Lambda function: custom data transformation + enrichment
├── API Gateway: call external API to enrich event
├── Step Functions: complex orchestration during enrichment
└── EventBridge connection: call third-party API

Targets (same as EventBridge rules):
├── SQS, SNS, Lambda, Step Functions, EventBridge bus, API Destination,
etc.

Example Pipes use case:
DynamoDB Stream (new orders)
  → Filter: only INSERT events for orders > $500
  → Enrichment: Lambda adds customer tier info from DynamoDB
  → Target: SQS queue for premium order processing

Without Pipes: need Lambda to poll DynamoDB Streams + custom filter +
enrich + route
With Pipes:    zero code for the plumbing — just configure source, filter,
enrich, target

aws pipes create-pipe \
  --name "high-value-order-pipe" \
  --role-arn arn:aws:iam::123:role/pipes-role \
  --source arn:aws:sqs:...:high-value-orders \
  --target arn:aws:sqs:...:premium-processing \
  --enrichment arn:aws:lambda:...:function:enrich-with-customer-data
```

---

## 🧩 EventBridge Scheduler

```
EventBridge Scheduler = managed scheduled job service
                        Replaces CloudWatch Events cron rules for
```

```
scheduling

Schedule types:
├── One-time:   run once at specific date/time
├── Rate-based: run every N minutes/hours/days
└── Cron:       standard cron expression (minute/hour/day/month/day-of-
week/year)

Universal targets: any AWS API action can be scheduled directly.
(vs CloudWatch Events which only supports predefined targets)

Schedule Lambda every 5 minutes:
aws scheduler create-schedule \
  --name "order-cleanup-5min" \
  --schedule-expression "rate(5 minutes)" \
  --target '{
    "Arn": "arn:aws:lambda:...:function:cleanup-orders",
    "RoleArn": "arn:aws:iam::123:role/scheduler-role"
  }'

Time zone support:
├── Supports all IANA time zones
└── Schedule at 9 AM EST without managing UTC offsets

⚠ EventBridge Scheduler is NOT the same as EventBridge cron rules.
   Scheduler: purpose-built, millions of concurrent schedules, time zones.
   EventBridge rules: event-based, one cron per rule, UTC only.
```

---

## 💬 Short Crisp Interview Answer

*"EventBridge routes events from sources to targets based on pattern matching rules. The default event bus receives all AWS service events automatically — EC2 state changes, GuardDuty findings, RDS snapshots. Custom event buses hold your application events, and partner buses receive events from SaaS providers like Datadog and Shopify. Rules define JSON patterns to match against events and route to up to 5 targets simultaneously. Patterns support exact matching, prefix, numeric ranges, exists checks, and anything-but. EventBridge Pipes are point-to-point integrations with a built-in filter and enrichment step between source (SQS, DynamoDB Streams, Kinesis) and target — eliminating boilerplate polling Lambda functions. API Destinations extend targets to any external HTTP endpoint with auth management."*

---

## ⚠ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| Pattern is subset matching | Only fields specified in the pattern are checked — unspecified fields are ignored |
| Max 5 targets per rule | One rule routes to up to 5 targets simultaneously. Need more → multiple rules |

| Gotcha | Detail |
|---|---|
| Event size limit | EventBridge max event size: 256KB |
| EventBridge is at-least-once | Duplicate event delivery is possible — targets must be idempotent |
| Cross-account needs resource policy | Must add resource-based policy on target bus to allow cross-account PutEvents |
| Default bus events are read-only | You cannot stop AWS services from publishing to the default bus |

---

# 9.5 SNS + SQS Fan-out Architecture

---

## 🟢 What It Is in Simple Terms

The fan-out pattern uses one SNS topic to deliver a single message to multiple SQS queues simultaneously. Each SQS queue has its own independent consumer that processes the message at its own pace. This combines SNS's broadcast capability with SQS's buffering, retry, and DLQ capabilities.

---

## 🔍 Why Fan-out Exists / What Problem It Solves

```
Problem: New order created. Must:
1. Process payment         (slow — external API, ~2 seconds)
2. Update inventory        (fast — database write, ~50ms)
3. Send confirmation email (async — fire and forget)
4. Update analytics        (batch — delay acceptable)

Option 1: Sequential processing in one Lambda:
→ Total time: ~2 seconds minimum
→ Payment failure blocks email send
→ All-or-nothing failure mode
→ Cannot scale each step independently

Option 2: Direct SNS to Lambda:
→ All Lambdas called simultaneously (good!)
→ But no buffering, no DLQ per subscriber
→ If one Lambda is throttled: SNS retries all subscribers (waste)
→ Cannot control processing rate per consumer

Option 3: SNS + SQS Fan-out (correct approach):
→ SNS delivers to 4 SQS queues simultaneously
→ Each queue has independent consumer
→ Each queue has independent DLQ
→ Each consumer scales independently
→ One consumer slow/failed = doesn't affect others
→ Queue buffers traffic spikes
```

---

## ⚙️ Fan-out Architecture

```
Fan-out Pattern:

                                ┌──────────────────────────────────────┐
                                │            SNS Topic                 │
Order Service                   │          "orders-created"            │
     │                          │                                      │
     └──────Publish─────────────►│                                      │
                                │                                      │
                                └──────┬──────┬──────┬──────┬──────┘
                                       │      │      │      │
                                       ▼      ▼      ▼      ▼
                                   SQS    SQS    SQS    SQS
                                payments inventory email-svc analytics
                                 queue   queue    queue    queue
                                   │      │      │      │
                                   ▼      ▼      ▼      ▼
                                Payment Inventory Email  Analytics
                                Worker  Worker    Lambda Lambda
                                   │      │
                                   ▼      ▼
                                Payment- Inventory-
                                  DLQ     DLQ

Each SQS queue:
├── Independent visibility timeout
├── Independent DLQ (failures don't affect other subscribers)
├── Independent consumer scaling
├── Independent retry configuration
└── Independent message retention
```

---

## 🧩 Full Fan-out Implementation

```
# Step 1: Create SNS topic
TOPIC_ARN=$(aws sns create-topic --name orders-created \
  --query 'TopicArn' --output text)

# Step 2: Create SQS queues for each consumer
aws sqs create-queue --queue-name payments-queue
aws sqs create-queue --queue-name payments-dlq
aws sqs create-queue --queue-name inventory-queue
aws sqs create-queue --queue-name inventory-dlq

# Step 3: Set DLQ on each queue
PAYMENTS_DLQ=$(aws sqs get-queue-attributes \
  --queue-url https://sqs.../payments-dlq \
```

```
  --attribute-names QueueArn --query 'Attributes.QueueArn' --output text)

aws sqs set-queue-attributes \
  --queue-url https://sqs.../payments-queue \
  --attributes '{
    "RedrivePolicy": "
{\"deadLetterTargetArn\":\"'"$PAYMENTS_DLQ"'\",\"maxReceiveCount\":\"3\"}"
  }'

# Step 4: Set SQS access policy to allow SNS delivery
# Each SQS queue needs a policy allowing SNS to send messages
aws sqs set-queue-attributes \
  --queue-url https://sqs.../payments-queue \
  --attributes '{
    "Policy": "{
      \"Statement\": [{
        \"Effect\": \"Allow\",
        \"Principal\": {\"Service\": \"sns.amazonaws.com\"},
        \"Action\": \"sqs:SendMessage\",
        \"Resource\": \"arn:aws:sqs:...:payments-queue\",
        \"Condition\": {
          \"ArnEquals\": {\"aws:SourceArn\": \"'"$TOPIC_ARN"'\"}
        }
      }]
    }"
  }'

# Step 5: Subscribe each SQS queue to SNS topic
# Payments: receives ALL events
aws sns subscribe \
  --topic-arn $TOPIC_ARN \
  --protocol sqs \
  --notification-endpoint arn:aws:sqs:...:payments-queue

# Inventory: only ORDER_CREATED events
aws sns subscribe \
  --topic-arn $TOPIC_ARN \
  --protocol sqs \
  --notification-endpoint arn:aws:sqs:...:inventory-queue \
  --attributes '{"FilterPolicy": "{\"eventType\": [\"ORDER_CREATED\"]}"}'
```

---

## 🧩 Fan-out with S3 Events

```
S3 bucket events fan-out pattern:
Single S3 upload → multiple processing pipelines

Problem: S3 only supports ONE event notification destination per event
type.
         (Cannot send s3:ObjectCreated to both Lambda AND SQS directly)
```

```
Solution: S3 → SNS topic → multiple SQS queues (fan-out)

S3 → SNS → SQS(image-resizer) + SQS(virus-scanner) + SQS(metadata-
extractor)

aws s3api put-bucket-notification-configuration \
  --bucket prod-uploads \
  --notification-configuration '{
    "TopicConfigurations": [{
      "TopicArn": "arn:aws:sns:...:file-uploaded",
      "Events": ["s3:ObjectCreated:*"]
    }]
  }'

⚠ S3 supports EventBridge natively now — can replace this pattern:
   Enable S3 → EventBridge integration → one EventBridge rule per consumer
   No SNS middle layer needed with EventBridge.
   But SNS fan-out remains valid and widely used in production.
```

---

## 🧩 Fan-out vs EventBridge vs Direct Lambda

```
┌──────────────────────┬──────────────────┬────────────────┬──────────────────┐
│ Feature              │ SNS+SQS Fan-out  │ EventBridge    │ Direct Lambda    │
├──────────────────────┼──────────────────┼────────────────┼──────────────────┤
│ Buffering            │ ✅ SQS buffers   │ ❌ No buffer   │ ❌ No buffer     │
│ DLQ per consumer     │ ✅ Yes           │ ✅ Yes         │ ❌ One DLQ only  │
│ Pattern matching     │ ✅ Filter policy │ ✅ Rich JSON   │ ❌ No filtering  │
│ Consumer rate ctrl   │ ✅ SQS rate ctrl │ ❌ No          │ ❌ No            │
│ Retry control        │ ✅ Per queue     │ ✅ Per target  │ ❌ No            │
│ Max consumers        │ Unlimited        │ 5 per rule     │ 1                │
│ Throughput           │ Very high        │ High           │ Lambda limit     │
│ Setup complexity     │ Medium           │ Low            │ Very low         │
└──────────────────────┴──────────────────┴────────────────┴──────────────────┘

Use SNS+SQS fan-out when:
├── Multiple consumers with independent processing rates
├── Need per-consumer DLQ and retry control
├── Consumers may be temporarily unavailable (SQS buffers)
```

```
  └── High-throughput event streaming to multiple systems

  Use EventBridge when:
  ├── Rich event routing with complex pattern matching
  ├── Integrating AWS services or SaaS partners
  ├── Event content-based routing
  └── Cross-account event routing
```

---

## 💬 Short Crisp Interview Answer

*"SNS+SQS fan-out delivers a single event to multiple independent consumers by subscribing multiple SQS queues to one SNS topic. Each SQS queue acts as a buffer with its own DLQ, retry logic, visibility timeout, and consumer scaling — complete independence between consumers. When one consumer is slow or failing, the others are completely unaffected. SNS filter policies let each queue receive only relevant events — inventory queue subscribes for ORDER_CREATED only, analytics queue subscribes for all events. This pattern is essential for S3 events where a bucket can only have one notification destination — S3 sends to SNS, SNS fans out to multiple SQS queues. The SQS buffering layer is the critical advantage over direct SNS-to-Lambda — it absorbs traffic spikes and provides per-consumer flow control."*

---

## ⚠ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| SQS access policy required | SQS queue must have resource policy allowing SNS:SendMessage or SNS delivery silently fails |
| SNS message wrapped in JSON | Message received in SQS has SNS envelope — parse the "Message" field inside |
| S3 single destination limit | S3 native notifications: one destination per event type. Use SNS or EventBridge to fan-out |
| FIFO fan-out requires FIFO | If ordering matters across fan-out: SNS FIFO → SQS FIFO queues only |

---

# 9.6 Kinesis Data Streams — Shards, Sequence Numbers, Enhanced Fan-out

---

## 🟢 What It Is in Simple Terms

Kinesis Data Streams is a real-time data streaming service for high-volume, ordered, replayable event streams. Unlike SQS (which deletes messages after consumption), Kinesis retains records for up to 365 days so multiple independent applications can read the same stream simultaneously, and failed applications can replay from any point.

## 🔍 Why It Exists / What Problem It Solves

```
SQS is built for work queues: task distribution, at-least-once processing.
Kinesis is built for data streams: ordered events, multiple consumers,
replay.

SQS limitations Kinesis solves:
├── SQS: message deleted after consumption → only ONE consumer per message
│   Kinesis: data retained → MULTIPLE consumers read the same stream
├── SQS: no guaranteed ordering across the full queue
│   Kinesis: strict ordering within each shard
├── SQS: no replay capability once message deleted
│   Kinesis: replay from any timestamp within retention window
└── SQS: no native high-throughput ingestion
    Kinesis: designed for millions of events/second

Use Kinesis when:
├── Multiple teams/applications need to process the same events
├── Real-time analytics (Flink, analytics apps)
├── Clickstream, IoT telemetry, application logs
└── Must replay historical data (compliance, reprocessing after bug fix)
```

## ⚙️ Kinesis Data Streams Architecture

```
Kinesis Data Streams Architecture:

Producers                   Kinesis Stream                Consumers

                          ┌──────────────────────────┐
                          │ Shard 1                  │
Web servers ────────────► │ seq: 0, 1, 2, 3, 4...    │──► Consumer App A
IoT devices ────────────► │ partition key hash range  │──► Consumer App B
(replay)
Mobile apps ────────────► ├──────────────────────────┤
                          │ Shard 2                  │
                          │ seq: 0, 1, 2, 3, 4...    │──► Consumer App A
                          ├──────────────────────────┤
                          │ Shard 3                  │
                          │ seq: 0, 1, 2, 3, 4...    │──► Consumer App A
                          └──────────────────────────┘
                                      │
                                Data retained
                                1 day (default)
                                up to 365 days

Stream: named, ordered sequence of records
Shard: unit of capacity — ONE shard provides:
  ├── Write: 1 MB/second OR 1,000 records/second (whichever reached first)
  └── Read:  2 MB/second total (shared among all standard consumers)
```

```
Record structure:
{
  "PartitionKey":   "device-123",    ← determines which shard
  "SequenceNumber":
"49590338271490256608559692538361571095921575989136588898",
  "Data":           "base64-encoded payload",
  "ApproximateArrivalTimestamp": 1705329000.0
}
```

---

## 🧩 Shards and Partition Keys

```
Partition Key → determines which shard a record goes to
└── Kinesis hashes partition key (MD5) → shard assignment
└── Same partition key → same shard → strict ordering guaranteed

Choosing partition key:
├── High cardinality: userId, deviceId, orderId → even distribution
└── Low cardinality: state, category → hot shards (uneven distribution)

Hot shard problem:
PartitionKey = "us-east-1" (all records from same region)
→ ALL records go to the same shard
→ That shard exceeds 1MB/s or 1000 records/s → throttling!
→ Fix: use userId or more specific identifier

Shard capacity math:
Requirement: 5 MB/s write, 10 MB/s read
Write shards needed: 5 MB/s ÷ 1 MB/s per shard = 5 shards
Read shards:         10 MB/s ÷ 2 MB/s per shard = 5 shards
→ Need 5 shards minimum

Scaling shards:
├── Split shard:  one shard → two shards (doubles capacity)
├── Merge shards: two shards → one shard (halves capacity)
└── Automatic: Kinesis On-Demand mode (auto-scales, no capacity planning)

Kinesis capacity modes:
├── Provisioned: manually configure shard count
│   $0.015/shard-hour + $0.014/GB ingested
└── On-Demand: auto-scales (up to 4 TB/day default)
    $0.04/GB ingested + $0.04/GB retrieved
    Best for: unpredictable or variable traffic

Retention:
├── Default: 24 hours
├── Extended: up to 365 days ($0.10/GB/month for extended)
└── Records available for replay during entire retention window
```

---

## 🧩 Consumers — Standard vs Enhanced Fan-out

```
Standard Consumer (GetRecords API — shared throughput):
├── Consumer polls Kinesis using GetRecords API call
├── All standard consumers SHARE the 2 MB/s read limit per shard
│   1 consumer: 2 MB/s available
│   5 consumers: 2/5 = 0.4 MB/s per consumer
├── GetRecords polling has up to 200ms latency
└── Up to 5 transactions/second GetRecords calls per shard

Enhanced Fan-out (SubscribeToShard API — dedicated throughput):
├── Each enhanced fan-out consumer gets 2 MB/s DEDICATED per shard
│   5 consumers with enhanced fan-out: 2 MB/s × 5 = 10 MB/s total!
├── Push-based: Kinesis pushes records to consumer (not polling)
├── ~70ms latency (vs 200ms polling)
└── Cost: $0.015/shard-hour per enhanced consumer + $0.013/GB retrieved

Standard vs Enhanced Fan-out:
┌──────────────────┬──────────────────┬──────────────────────┐
│ Feature          │ Standard         │ Enhanced Fan-out     │
├──────────────────┼──────────────────┼──────────────────────┤
│ Throughput       │ 2 MB/s shared    │ 2 MB/s per consumer  │
│ Latency          │ ~200ms (polling) │ ~70ms (push)         │
│ API              │ GetRecords poll  │ SubscribeToShard push│
│ Cost             │ Included         │ $0.015/shard/hr extra│
│ Max consumers    │ 5                │ 20                   │
└──────────────────┴──────────────────┴──────────────────────┘

Use enhanced fan-out when:
├── Multiple independent consumers need full 2 MB/s each
├── Low latency processing required (<100ms)
└── More than 2-3 consumers on the same stream
```

---

## 🧩 Sequence Numbers and Iterators

```
Sequence Number:
├── Unique identifier for each record within a shard
├── Monotonically increasing within a shard
├── Format: 49-digit integer (encodes shard + timestamp + offset)
└── Use for: checkpointing — "I processed up to this sequence number"

Shard Iterators (where to start reading):
├── TRIM_HORIZON:    oldest available record (beginning of retention
window)
├── LATEST:          only new records from now forward
├── AT_SEQUENCE_NUMBER:  start at specific sequence number
├── AFTER_SEQUENCE_NUMBER: start after specific sequence number
└── AT_TIMESTAMP:    start from specific timestamp
```

```
Checkpointing pattern (prevent reprocessing):
1. Consumer reads records from shard
2. Processes records successfully
3. Saves sequence number to DynamoDB (or uses KCL)
4. On restart: reads last checkpoint sequence number
5. Resumes from AT_SEQUENCE_NUMBER = last checkpoint

KCL (Kinesis Client Library):
├── Manages shard assignment across multiple consumer instances
├── Handles checkpointing automatically to DynamoDB
├── Handles shard splits/merges automatically
└── Uses DynamoDB for coordination (one row per shard)
    ⚠ KCL creates a DynamoDB table — ensure DynamoDB capacity
```

```python
# Python consumer using Kinesis SDK
import boto3, base64, time

kinesis = boto3.client('kinesis')

# Get shard iterator (start from beginning)
shard_iter = kinesis.get_shard_iterator(
    StreamName='prod-events',
    ShardId='shardId-000000000000',
    ShardIteratorType='TRIM_HORIZON'
)['ShardIterator']

# Poll for records
while True:
    response = kinesis.get_records(
        ShardIterator=shard_iter,
        Limit=100
    )

    for record in response['Records']:
        data = base64.b64decode(record['Data'])
        seq  = record['SequenceNumber']
        print(f"Processing seq={seq}: {data}")
        # TODO: save checkpoint to DynamoDB

    shard_iter = response['NextShardIterator']

    if not response['Records']:
        time.sleep(1)  # avoid hot polling
```

---

## 💬 Short Crisp Interview Answer

*"Kinesis Data Streams is a real-time ordered stream with per-shard retention (up to 365 days). Each shard provides 1 MB/s write and 2 MB/s read. Partition key determines which shard a record goes to — same partition key guarantees ordering. Standard consumers share the 2 MB/s read limit per shard. Enhanced*

*fan-out gives each consumer a dedicated 2 MB/s with ~70ms push-based delivery instead of polling. The key Kinesis advantages over SQS: multiple consumers reading the same stream independently, replay from any point in retention window, and strict ordering within a shard. Kinesis On-Demand mode removes capacity planning. The main gotcha: hot shards from low-cardinality partition keys — every record with the same key hits the same shard and can exceed its limits."*

---

## ⚠ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| Hot shard from low-cardinality PK | All records with same partition key → one shard → exceeds 1MB/s or 1000 records/s limit |
| Standard consumer shared throughput | 5 consumers share 2 MB/s per shard = 0.4 MB/s each — use enhanced fan-out |
| Shard iterator expires | GetShardIterator tokens expire after 5 minutes if not used |
| KCL needs DynamoDB capacity | KCL creates a DynamoDB table for coordination — provision adequate DynamoDB capacity |
| GetRecords max per shard | Max 5 GetRecords calls/second per shard — polling too fast causes throttling |
| Retention cost | Extended retention beyond 24 hours costs $0.10/GB/month — budget carefully |

---

# 9.7 Kinesis Firehose — Delivery Streams, Transformation, Buffering

---

## 🟢 What It Is in Simple Terms

Kinesis Firehose (now called Amazon Data Firehose) is a fully managed service that captures, transforms, and delivers streaming data to storage destinations like S3, Redshift, and OpenSearch. Unlike Kinesis Data Streams (which you must consume yourself), Firehose handles all the delivery logic automatically — you just configure the destination.

---

## 🔍 Kinesis Data Streams vs Kinesis Firehose

```
┌──────────────────┬──────────────────┬──────────────────┐
│ Feature          │ Data Streams     │ Firehose         │
├──────────────────┼──────────────────┼──────────────────┤
│ Consumer         │ You build consumers │ Fully managed    │
│ Delivery         │ Real-time (<70ms)  │ Buffered (60s-15min) │
│ Destinations     │ Any (custom code)  │ S3, Redshift,    │
```

```
│                        │                        │ OpenSearch, Splunk    │
│ Data transformation    │ Custom (in consumer)   │ Lambda (built-in)     │
│ Data format            │ Any                    │ Parquet/ORC convert   │
│ Retention              │ 1-365 days             │ No retention (stream)│
│ Replay                 │ ✅ Yes                 │ ❌ No (after deliver)│
│ Multiple consumers     │ ✅ Yes                 │ ❌ No (one dest)      │
│ Scaling                │ Manual or on-demand    │ Fully automatic       │
│ Operational overhead   │ Higher                 │ Near zero             │
└────────────────────────┴────────────────────────┴───────────────────────┘

Use Firehose when:
├── Delivering data to S3, Redshift, OpenSearch, Splunk
├── Want zero consumer management (fully managed delivery)
└── Buffered delivery (near-real-time, not millisecond)

Use Data Streams when:
├── Multiple independent consumers on the same stream
├── Need <1 second latency (true real-time)
└── Need replay capability
```

---

## ⚙️ Firehose Architecture

```
Firehose Delivery Stream:

Sources:
├── Direct PUT: applications write directly to Firehose
├── Kinesis Data Streams: Firehose reads from KDS
├── MSK (Managed Kafka): Firehose reads from Kafka topic
├── CloudWatch Logs: subscription filter → Firehose
└── EventBridge: rule target → Firehose

Data flow:
Records arrive
    │
    ▼ [Optional: Lambda transformation]
Buffer fills (size OR time threshold)
    │
    ▼
Deliver batch to destination

Destinations:
├── Amazon S3 (most common)
├── Amazon Redshift (via S3 COPY command)
├── Amazon OpenSearch Service
├── Splunk
└── Custom HTTP endpoint (any destination)

S3 destination with prefix:
s3://my-bucket/events/year=2024/month=01/day=15/hour=14/
```

```
S3 dynamic partitioning:
Use record field values as S3 prefix components:
prefix: "orders/region=!{partitionKeyFromQuery:region}/year=!
{timestamp:yyyy}/"
→ Separate S3 prefix per region, automatically partitioned by year
```

---

## 🧩 Buffering

```
Buffering = Firehose accumulates records then delivers as a batch
            Two thresholds — whichever is reached first triggers delivery:

├── Buffer size:     deliver when buffer reaches N MB (1-128 MB)
└── Buffer interval: deliver when buffer has been collecting for N seconds
                     (60-900 seconds)

Examples:
Buffer: 5 MB or 60 seconds → batch delivered when either is reached
Buffer: 128 MB or 900 seconds → large batches, fewer S3 objects, lower
cost

Small buffer (1 MB / 60 seconds):
├── More frequent deliveries → lower latency to destination
├── More S3 objects → higher S3 PUT costs + harder to query
└── Use when: near-real-time visibility required

Large buffer (128 MB / 900 seconds):
├── Fewer, larger deliveries → higher latency to destination
├── Fewer S3 objects → lower S3 costs + better Athena query performance
└── Use when: batch analytics, cost optimization

Cost: $0.029/GB of data ingested
No shard management, no consumer code, no scaling decisions.

⚠ Firehose delivers at least once — duplicates possible.
   Downstream processing must handle duplicates.
```

---

## 🧩 Lambda Transformation

```
Lambda transformation = process records mid-stream before delivery

Use cases:
├── Filter: drop records that don't match criteria (reduce storage cost)
├── Enrich: add fields from lookup (DynamoDB, API call)
├── Transform: convert CSV → JSON, JSON → Parquet
├── Mask: remove PII before storing (GDPR compliance)
└── Parse: extract fields from unstructured log lines
```

```
Lambda receives batches of records from Firehose:
{
  "records": [
    {
      "recordId":
"49590338271490256608559692538361571095921575989136588898",
      "approximateArrivalTimestamp": 1705329000000,
      "data": "base64-encoded-record"
    }
  ]
}

Lambda must return ALL records with one of three statuses:
├── "Ok":              record processed successfully → deliver to
destination
├── "Dropped":         record intentionally discarded → not delivered
└── "ProcessingFailed": record failed processing → sent to error S3 prefix

Lambda transformation response:
{
  "records": [
    {
      "recordId": "49590338...",
      "result": "Ok",           ← or "Dropped" or "ProcessingFailed"
      "data": "base64-encoded-transformed-record"
    }
  ]
}
```

```python
# Example Lambda transformation: filter and enrich
import base64, json

def handler(event, context):
    output = []

    for record in event['records']:
        payload = json.loads(base64.b64decode(record['data']))

        # Filter: drop records from internal health checks
        if payload.get('source') == 'health-check':
            output.append({
                'recordId': record['recordId'],
                'result': 'Dropped',
                'data': record['data']
            })
            continue

        # Enrich: add processing timestamp
        payload['processedAt'] = '2024-01-15T14:30:00Z'
        payload['environment'] = 'prod'
```

```
        # Transform back to base64
        output.append({
            'recordId': record['recordId'],
            'result': 'Ok',
            'data': base64.b64encode(
                json.dumps(payload).encode('utf-8')
            ).decode('utf-8')
        })

    return {'records': output}
```

---

## 🧩 Format Conversion

```
Firehose native format conversion (no Lambda needed):

Input formats: JSON
Output formats: Apache Parquet or Apache ORC (columnar formats)

Why convert to Parquet:
├── 4-10x less storage (columnar compression)
├── 10-100x faster Athena queries (reads only needed columns)
└── Zero application code changes

Conversion requires:
├── AWS Glue Data Catalog schema for the data
└── Firehose reads schema → converts JSON to Parquet on the fly

aws firehose create-delivery-stream \
  --delivery-stream-name prod-events-to-s3 \
  --extended-s3-destination-configuration '{
    "RoleARN": "arn:aws:iam::123:role/firehose-role",
    "BucketARN": "arn:aws:s3:::my-data-lake",
    "Prefix": "events/year=!{timestamp:yyyy}/month=!{timestamp:MM}/",
    "BufferingHints": {"SizeInMBs": 128, "IntervalInSeconds": 300},
    "DataFormatConversionConfiguration": {
      "InputFormatConfiguration": {
        "Deserializer": {"OpenXJsonSerDe": {}}
      },
      "OutputFormatConfiguration": {
        "Serializer": {"ParquetSerDe": {"Compression": "SNAPPY"}}
      },
      "SchemaConfiguration": {
        "DatabaseName": "prod_db",
        "TableName": "events",
        "RoleARN": "arn:aws:iam::123:role/glue-role"
      }
    }
  }'
```

---

## 💬 Short Crisp Interview Answer

*"Kinesis Firehose is a fully managed delivery service — you don't write consumers, manage scaling, or handle delivery logic. Records buffer until either a size threshold or time interval is reached, then deliver as a batch to S3, Redshift, OpenSearch, or Splunk. Lambda transformation processes records mid-stream for filtering, enrichment, or PII masking before delivery. Format conversion natively converts JSON to Parquet using a Glue schema — zero code, dramatically better compression and query performance in Athena. The key trade-offs versus Data Streams: Firehose is simpler with zero management but has 60-second minimum latency and no replay capability. Choose Data Streams for real-time multiple-consumer processing; choose Firehose for simple delivery pipelines to storage destinations."*

---

## ⚠️ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| Minimum 60-second latency | Firehose buffer interval minimum is 60 seconds — not true real-time |
| Lambda must return ALL records | Lambda transformation must return a result for every input record — missing records cause errors |
| ProcessingFailed records | Failed Lambda records sent to error S3 prefix — monitor this prefix for issues |
| Firehose delivers at-least-once | Duplicates are possible — design downstream consumers to be idempotent |
| Redshift delivery via S3 COPY | Firehose delivers to S3 first, then issues COPY to Redshift — not a direct stream insert |
| Dynamic partitioning has cost | Dynamic partitioning adds $0.03/GB — evaluate cost vs query performance benefit |

---

# 9.8 MSK — Managed Kafka vs Kinesis, Use Cases

---

## 🟢 What It Is in Simple Terms

MSK (Managed Streaming for Apache Kafka) is AWS's fully managed Apache Kafka service. Kafka is the industry-standard open-source distributed event streaming platform — MSK removes the operational burden of running Kafka clusters (broker management, patching, scaling) while keeping full Kafka API compatibility.

---

## 🔍 Why MSK Exists / What Problem It Solves

```
Running Kafka yourself (on EC2):
├── Provision and manage Kafka broker EC2 instances
├── Manage Apache ZooKeeper (coordination service)
├── Handle broker failures, rebalancing, upgrades
├── Configure storage, networking, monitoring
└── On-call for 3am Kafka cluster issues

MSK removes all of this — AWS manages:
├── Broker provisioning and replacement
├── ZooKeeper (or KRaft mode — no ZooKeeper)
├── Automatic broker recovery on failure
├── Storage expansion
├── Security patching
└── CloudWatch metrics + logging integration

You manage:
├── Topics (create, configure, partition count, replication factor)
├── Consumer groups and offset management
└── Schema management (optionally with Glue Schema Registry)
```

---

## 🧩 Kafka Core Concepts (for MSK context)

```
Kafka Architecture:

Producers                Kafka Cluster (MSK)              Consumers
                      ┌──────────────────────────────┐
                      │ Topic: "orders"              │
Web App ──────────►   │ Partition 0: [r0,r1,r2...] │──► Consumer Group A
Mobile ──────────►    │ Partition 1: [r0,r1,r2...] │──► Consumer Group A
IoT ──────────────►   │ Partition 2: [r0,r1,r2...] │──► Consumer Group B
                      │                              │    (replay from offset
0)
                      │ Replication Factor: 3        │
                      │ (3 copies of each partition)│
                      └──────────────────────────────┘


Key concepts:
├── Broker:       Kafka server instance (MSK manages these)
├── Topic:        named stream of records (like Kinesis stream)
├── Partition:    ordered immutable log (like Kinesis shard)
├── Offset:       position within a partition (like sequence number)
├── Producer:     sends records with a key (key → partition assignment)
├── Consumer:     reads records from partitions
└── Consumer Group: multiple consumers sharing a topic's partitions
    (each partition assigned to ONE consumer in the group)
    (different groups get independent copies of all records)

Retention:
├── Time-based: keep records for N days (default 7 days)
```

```
├── Size-based:  keep records up to N bytes per partition
└── Log compaction: keep only latest value per key (for state tables)
```

---

## 🧩 MSK vs Kinesis Data Streams

```
┌──────────────────────┬──────────────────────┬──────────────────────┐
│ Feature              │ MSK (Kafka)          │ Kinesis Data Streams │
├──────────────────────┼──────────────────────┼──────────────────────┤
│ Ecosystem            │ Kafka ecosystem      │ AWS-native only      │
│                      │ (Flink, Spark, etc.) │                      │
│ Partition limit      │ Unlimited (practical)│ Hard limit per acct  │
│ Scaling              │ Manual (add brokers) │ On-Demand or manual  │
│ Consumer groups      │ Unlimited groups     │ Limited consumers    │
│ Message size         │ Default 1MB          │ 1MB hard limit       │
│ Retention            │ Unlimited            │ 365 days max         │
│ Security             │ IAM, mTLS, SASL/SCRAM│ IAM only             │
│ Multi-region         │ MirrorMaker2         │ Native cross-region  │
│ Operational effort   │ More (topic config)  │ Less (managed)       │
│ Cost model           │ Per broker-hour      │ Per shard-hour       │
│ Vendor lock-in       │ No (open source)     │ AWS-specific         │
│ Schema registry      │ Confluent / Glue     │ Glue                 │
└──────────────────────┴──────────────────────┴──────────────────────┘

Choose MSK when:
├── Already using Kafka in on-premises or other clouds (lift and shift)
├── Need Kafka ecosystem tools (Kafka Streams, Kafka Connect, ksqlDB)
├── Need Flink or Spark Structured Streaming with Kafka connector
├── Want to avoid vendor lock-in (open-source protocol)
├── Need more than 365 days retention
└── Complex topic-level configurations (compaction, custom retention)

Choose Kinesis Data Streams when:
├── Pure AWS environment (no Kafka expertise needed)
├── Tighter AWS service integration (Lambda ESM, Firehose native)
├── Less operational overhead preferred
├── On-Demand scaling is important
└── Team comfortable with AWS-native services
```

---

## 🧩 MSK Configuration

```
MSK Cluster types:
├── Provisioned: choose broker instance type and count
│   Instance types: kafka.m5.large → kafka.m5.24xlarge
│   Minimum: 3 brokers (for replication factor 3)
└── MSK Serverless: pay-per-use, auto-scales
    No broker management, no capacity planning
    Best for: dev/test, spiky workloads
```

```
Topic configuration:
aws kafka create-topic \
  --cluster-arn arn:aws:kafka:... \
  --topic-name orders \
  --partitions 12 \
  --replication-factor 3 \
  --config '{"retention.ms": "604800000",  # 7 days
             "min.insync.replicas": "2",
             "compression.type": "lz4"}'

Key topic settings:
├── partitions:          controls parallelism (more = more consumers)
│   Rule: partitions ≥ expected consumer group members
├── replication-factor:  copies of each partition across brokers
│   Minimum 3 in production (MSK default: 3)
├── retention.ms:        how long to keep records
├── min.insync.replicas: minimum replicas that must acknowledge a write
│   Set to 2 for production (leader + at least 1 follower)
└── compression.type:    lz4, snappy, gzip, zstd

MSK Connect (managed Kafka Connect):
├── Run Kafka connectors without managing infrastructure
├── Source connectors: database change data capture → Kafka
│   (Debezium for MySQL, PostgreSQL CDC)
├── Sink connectors: Kafka → S3, OpenSearch, databases
└── Fully managed worker scaling
```

---

## 🧩 MSK Serverless

```
MSK Serverless:
├── No cluster sizing decisions
├── Automatically scales throughput capacity
├── Pay per MB/hour of partition storage
├── Supports IAM authentication only (no mTLS, no SASL/SCRAM)
└── Pricing: $0.75/cluster-hour + $0.005/GB throughput

Use MSK Serverless when:
├── Variable or unpredictable workloads
├── Dev and staging environments
├── Don't want to size Kafka brokers
└── Teams new to Kafka (simpler operation)

Limitations:
├── IAM auth only (no SASL/SCRAM, no mTLS)
├── Limited topic configuration options
└── Higher cost at consistent high throughput vs provisioned
```

---

## 💬 Short Crisp Interview Answer

*"MSK is AWS-managed Apache Kafka — AWS handles broker provisioning, replacement, ZooKeeper, and patching while you manage topics, consumer groups, and schema. Choose MSK when migrating existing Kafka workloads (same Kafka protocol), needing ecosystem tools like Kafka Streams or Kafka Connect, or requiring vendor portability with open-source APIs. Choose Kinesis Data Streams for pure AWS environments with tighter native integration (Lambda ESM, Firehose), simpler operations, and On-Demand scaling. The key Kafka advantage is unlimited consumer groups with independent offsets and no consumer group limit — hundreds of different applications can read the same topic independently. MSK Serverless removes capacity planning for variable workloads. MSK Connect runs managed Kafka connectors for CDC pipelines from databases into Kafka."*

---

## ⚠️ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| min.insync.replicas | Must set to 2 in production. Default 1 = single point of failure for writes |
| Partition count is permanent | Cannot reduce partition count after topic creation — only increase |
| Consumer group rebalancing | Adding/removing consumers triggers partition rebalance — brief processing pause |
| MSK Serverless = IAM only | MSK Serverless doesn't support SASL/SCRAM or mTLS authentication |
| Broker storage auto-expansion | Enable auto-scaling on storage or manually monitor — full brokers stop accepting writes |
| ZooKeeper vs KRaft | Newer MSK versions support KRaft mode (no ZooKeeper) — prefer KRaft for new clusters |

---

# 9.9 SQS — Exactly-Once Processing Patterns, Poison Pill Handling

---

## 🟢 What It Is in Simple Terms

SQS Standard queues deliver at-least-once — the same message can arrive more than once. Exactly-once processing requires application-level design to handle duplicates safely. Poison pill messages are malformed messages that always fail processing and must be detected and isolated before they block or monopolize your consumers.

---

## 🔍 Why This Matters

```
At-least-once delivery causes:
├── Duplicate charges (payment processed twice)
├── Duplicate emails (customer receives 2 confirmation emails)
├── Data corruption (same DB record updated twice with conflicting state)
└── Inventory errors (item reserved twice from the same stock)

In production EVERY SQS consumer must be designed to handle duplicates.
"My code never duplicates" — SQS will deliver it twice. Design for it.

Poison pill causes:
├── Message with malformed JSON → consumer crashes on parse → infinite
retry
├── Message referencing deleted resource → consumer fails → infinite retry
├── Message too large for downstream → consumer fails → infinite retry
└── Without DLQ: message retried forever → consumer CPU wasted
    With DLQ:    after N retries → DLQ → investigate → fix → replay
```

---

## 🧩 Idempotency — The Foundation of Exactly-Once

```
Idempotency = processing the same message twice = same result as once

Every SQS consumer should be idempotent by design.

Idempotency strategies:

1. Natural idempotency (best — no extra infrastructure):
   Operation itself is naturally idempotent:
   ├── SET user.email = "new@email.com" → idempotent (same result each
time)
   ├── UPDATE inventory SET reserved = true WHERE id = 123 → idempotent
   └── PUT /users/123 {email: "new@email.com"} → idempotent (REST PUT)

   Non-idempotent (requires deduplication):
   ├── INSERT INTO orders (amount, ...) → creates duplicate row each time
   ├── charge_credit_card(amount=99.99) → charges twice!
   └── send_email(to="user@email.com") → sends duplicate email
    Most payment providers (Stripe, PayPal, Razorpay) support an
Idempotency-Key header. Passing orderId as this key means even if the API
is called twice, the customer is charged only once. This is the cleanest
solution when the downstream API supports it — no deduplication table
needed.

2. Database deduplication (idempotency key in DB):
CREATE TABLE processed_messages (
    message_id VARCHAR(255) PRIMARY KEY,  ← SQS MessageId
    processed_at TIMESTAMP,
    result TEXT
);
```

```
def process_message(message):
    message_id = message['MessageId']

    # Atomic check-and-insert
    try:
        db.execute(
            "INSERT INTO processed_messages (message_id, processed_at)
VALUES (?, NOW())",
            [message_id]
        )
    except UniqueConstraintViolation:
        # Already processed - skip silently
        print(f"Duplicate detected: {message_id}, skipping")
        return

    # First time seeing this message - process normally
    do_actual_work(message)

3. DynamoDB conditional writes (distributed, no DB needed):
def process_message(message):
    message_id = message['MessageId']

    try:
        # Conditional put: only succeed if message_id not seen before
        table.put_item(
            Item={'messageId': message_id, 'processedAt': time.time()},
            ConditionExpression='attribute_not_exists(messageId)'
        )
    except table.meta.client.exceptions.ConditionalCheckFailedException:
        print(f"Duplicate: {message_id}")
        return

    do_actual_work(message)

4. FIFO queue deduplication (SQS-native):
SQS FIFO with MessageDeduplicationId:
├── Same MessageDeduplicationId within 5-minute window → silently dropped
└── But FIFO has 3,000 TPS limit - not suitable for all workloads
```

---

## 🧩 Poison Pill Handling

```
Poison Pill = message that ALWAYS fails processing
              Causes infinite retry loop without intervention

Poison pill scenarios:
├── Malformed JSON:          {"amount": "invalid", → parse error every
time
├── Schema version mismatch: consumer expects v2 schema, message is v1
├── Missing resource:        message references deleted order ID
├── External service bug:    downstream API consistently rejects specific
```

```
payload
└── Message too large:       downstream system has size limit below 256KB

Detection pattern:
Message received → processing attempt fails → message becomes visible
→ Consumer receives same message again → fails again → visible again
Without DLQ: infinite loop → consumer time/CPU wasted on unprocessable
message
With DLQ:    after maxReceiveCount (e.g., 3) → moved to DLQ

DLQ best practices:
├── Set maxReceiveCount = 3-5 (low enough to detect fast, high enough for
transient failures)
├── Set DLQ retention = 14 days (time to investigate)
├── Alarm on: ApproximateNumberOfMessagesVisible > 0 in DLQ
└── Never let DLQ fill silently — page someone immediately

Distinguishing transient vs poison pill failures:
├── Transient: network timeout, DB connection error → retries usually
succeed
├── Poison pill: consistent failure → always moves to DLQ
└── Detection: if message reaches DLQ → assume poison pill → investigate

DLQ investigation workflow:
1. Alarm fires: message in DLQ
2. Read message from DLQ: aws sqs receive-message --queue-url .../orders-
dlq
3. Inspect message body and attributes
4. Identify root cause:
   ├── Bug in consumer code → fix code → replay
   ├── Malformed message → log for audit → discard
   └── Missing resource → restore resource → replay
5. Replay: aws sqs start-message-move-task \
     --source-arn arn:...dlq --destination-arn arn:...source-queue
```

---

## 🧩 Advanced Exactly-Once Patterns

```
Pattern 1: Outbox Pattern (database + SQS consistency)

Problem: Service updates DB and sends SQS message — these two operations
         are not atomic. DB update succeeds but SQS send fails →
inconsistency.

Solution: Outbox table in same database transaction:
1. BEGIN TRANSACTION
2. UPDATE orders SET status = 'CONFIRMED' WHERE id = 456
3. INSERT INTO outbox (payload, sent) VALUES ('{...}', false)
4. COMMIT TRANSACTION
5. Separate background process polls outbox table → sends to SQS → marks
sent=true
```

```
Both DB update and SQS message delivery are guaranteed (retried
separately).
Even if service crashes between step 4 and 5, outbox records resend on
restart.

Pattern 2: Saga with SQS (distributed transaction without 2PC)

Multi-step workflow (e.g., book flight + hotel + car):
Step 1: Reserve flight → success → send "flight-reserved" to SQS
Step 2: Reserve hotel  → FAIL   → send "hotel-failed" to SQS
Step 3: Saga receives "hotel-failed" → sends compensating "cancel-flight"
to SQS
Step 4: Cancel flight  → success → saga complete (no inconsistency)

Each step in SQS queue with its own consumer.
Compensating transactions handle rollback.
No distributed lock, no 2PC, fully asynchronous.

Pattern 3: Message ordering with SQS FIFO + MessageGroupId

All events for the same order must process in order:
- ORDER_CREATED → ORDER_PAYMENT_RECEIVED → ORDER_SHIPPED

SQS FIFO with MessageGroupId = orderId:
├── All events for orderId=456 → same group → strict FIFO order
├── Events for orderId=789 → different group → processed in parallel
└── Parallelism = number of distinct MessageGroupIds
    (N different orderIds → N parallel processing streams)

def send_order_event(order_id, event_type, payload):
    sqs.send_message(
        QueueUrl=FIFO_QUEUE_URL,
        MessageBody=json.dumps(payload),
        MessageGroupId=str(order_id),           # order-level ordering
        MessageDeduplicationId=f"{order_id}:{event_type}:{time.time()}"
    )
```

---

## 🧩 Circuit Breaker Pattern with SQS

```
Problem: Downstream service is overwhelmed.
         SQS consumers keep pulling and calling the downstream service.
         → Makes the downstream service worse (overload amplification).

Solution: Circuit breaker — stop processing when downstream is unhealthy.

Implementation:
import boto3, time

sqs = boto3.client('sqs')
```

```
failure_count = 0
circuit_open = False
circuit_open_until = 0

def process_with_circuit_breaker():
    global failure_count, circuit_open, circuit_open_until

    # Check circuit state
    if circuit_open and time.time() < circuit_open_until:
        print("Circuit OPEN — skipping processing, backing off")
        time.sleep(30)  # don't poll SQS while circuit open
        return

    if circuit_open and time.time() >= circuit_open_until:
        print("Circuit HALF-OPEN — trying one request")
        circuit_open = False
        failure_count = 0

    response = sqs.receive_message(
        QueueUrl=QUEUE_URL,
        WaitTimeSeconds=20
    )

    for message in response.get('Messages', []):
        try:
            call_downstream_service(message)
            failure_count = 0  # reset on success
            sqs.delete_message(
                QueueUrl=QUEUE_URL,
                ReceiptHandle=message['ReceiptHandle']
            )
        except DownstreamException:
            failure_count += 1
            if failure_count >= 5:
                circuit_open = True
                circuit_open_until = time.time() + 60  # open for 1 minute
                print("Circuit OPENED — too many failures")
            # Don't delete — let message become visible again (retry
later)
```

---

## 💬 Short Crisp Interview Answer

*"SQS Standard delivers at-least-once, so all consumers must be idempotent by design. The three main strategies: natural idempotency using PUT/SET operations instead of INSERT/charge, database deduplication using a processed_messages table with MessageId as primary key and conditional insert, and FIFO queue deduplication using MessageDeduplicationId for strict exactly-once within 5 minutes. Poison pills are messages that always fail — without a DLQ they create infinite retry loops wasting consumer resources. With DLQ and maxReceiveCount=3, after 3 failures the message moves to DLQ, alerts fire, ops investigates, and after fixing the consumer, messages replay with the message-move-task API. Always*

*alarm on any DLQ depth > 0. The outbox pattern solves dual-write consistency: write to DB and outbox table in the same transaction, separate process sends from outbox to SQS."*

---

## ⚠ Tricky Edge Cases / Gotchas

| Gotcha | Detail |
|---|---|
| MessageId uniqueness | SQS MessageId is unique per send, not content — re-sent messages get new IDs |
| Deduplication window (FIFO) | FIFO deduplication only lasts 5 minutes — after that, same deduplication ID is accepted again |
| DLQ doesn't pause source queue | Messages continue flowing to source queue while DLQ grows — monitor both |
| Poison pill masking transient issues | If maxReceiveCount is too high (e.g., 10), transient issues look like poison pills and trigger alerts |
| Idempotency key TTL | Processed_messages table grows forever — implement TTL cleanup (e.g., delete entries older than 7 days) |
| FIFO GroupId = single consumer | Each MessageGroupId is processed by ONE consumer at a time — high-volume single group = bottleneck |

---

# 🔗 Category 9 — Full Connections Map

```
MESSAGING connects to:

SQS
├── Lambda              → Event Source Mapping (Lambda polls SQS)
├── SNS                 → SNS delivers messages into SQS queues (fan-out)
├── EventBridge         → EB rule target = SQS queue
├── Kinesis Firehose    → SQS can trigger Firehose via EventBridge Pipes
├── EC2 / ECS           → custom consumers poll SQS
├── CloudWatch Alarms   → alarm on queue depth, DLQ message count
└── KMS                 → SQS queue encryption with KMS key

SNS
├── SQS                 → primary subscriber (fan-out pattern)
├── Lambda              → direct invocation from SNS
├── HTTP/HTTPS          → push to any endpoint
├── SES                 → SNS → email delivery
├── CloudWatch Alarms   → alarm actions publish to SNS topic
├── EventBridge         → EB rule target = SNS topic
└── KMS                 → SNS message encryption with KMS key

EventBridge
├── All AWS services    → default bus receives all AWS service events
```

```
  ├── Lambda              → most common target for event-driven logic
  ├── SQS                 → buffer events with queue
  ├── Step Functions      → start complex workflows from events
  ├── Kinesis Firehose    → stream events to S3/OpenSearch
  ├── API Destinations    → call third-party SaaS (PagerDuty, Slack)
  ├── EventBridge Pipes   → point-to-point with filter + enrichment
  └── EventBridge Scheduler → scheduled jobs (cron + one-time)

  Kinesis Data Streams
  ├── Lambda              → Event Source Mapping (Lambda consumes shards)
  ├── Kinesis Firehose    → Firehose reads from KDS for managed delivery
  ├── EMR / Flink / Spark → analytics frameworks consume stream
  ├── KCL consumers       → custom consumer apps with checkpointing
  ├── EventBridge Pipes   → KDS as source for pipe routing
  └── CloudWatch          → stream metrics (IncomingRecords,
  GetRecords.Latency)

  Kinesis Firehose
  ├── S3                  → primary destination (data lake)
  ├── Redshift            → via S3 COPY command
  ├── OpenSearch          → for log/event search
  ├── Splunk              → security/log analytics
  ├── Lambda              → mid-stream transformation
  ├── Glue                → schema for format conversion (JSON → Parquet)
  └── CloudWatch Logs     → subscription filter → Firehose

  MSK (Kafka)
  ├── MSK Connect         → Kafka Connect source/sink connectors
  ├── Lambda              → Event Source Mapping for Kafka topics
  ├── Glue Schema Registry → schema validation and evolution
  ├── EMR / Flink         → Kafka-native stream processing
  ├── Kinesis Firehose    → Firehose reads from MSK
  └── CloudWatch          → broker metrics + topic metrics
```

---

## 📌 Quick Reference — Interview Cheat Sheet

| Topic | Key Fact |
|---|---|
| SQS Standard delivery | At-least-once. Duplicates WILL happen. Consumers must be idempotent |
| SQS Standard ordering | Best-effort. Usually in order but NOT guaranteed |
| SQS FIFO ordering | Strict FIFO within MessageGroupId. Different groups processed in parallel |
| SQS FIFO throughput | 300 TPS without batching, 3,000 TPS with batching |
| SQS FIFO deduplication | MessageDeduplicationId deduplication window = 5 minutes |
| Visibility timeout | Must be longer than max processing time or duplicates occur |
| Lambda SQS timeout rule | Lambda visibility timeout ≥ 6 × Lambda function timeout |
| DLQ maxReceiveCount | After N failures, message moves to DLQ. Alert on any DLQ depth > 0 |

| Topic | Key Fact |
|---|---|
| DLQ same queue type | Standard queue DLQ must be Standard. FIFO queue DLQ must be FIFO |
| Long polling | ReceiveMessageWaitTimeSeconds=20. Always use in production. |
| Short polling default | Default ReceiveMessageWaitTimeSeconds=0 — wastes API calls |
| Delay queue max | 15 minutes (900 seconds) |
| Per-message delay | Available on Standard queues only. NOT on FIFO queues |
| Message attributes | Max 10 per message. Count toward 256KB total size limit |
| SNS push-based | SNS pushes to subscribers. SQS consumers pull |
| SNS FIFO subscribers | FIFO topic only delivers to SQS FIFO queues — no Lambda/HTTP/email |
| SNS message size | 256KB maximum per message |
| SNS filter policy scope | MessageAttributes (default) or MessageBody (filter on body fields) |
| Fan-out SQS policy | SQS queue needs resource policy allowing sns:SendMessage or delivery fails |
| SNS message in SQS | Message arrives wrapped in SNS envelope JSON — parse the "Message" field |
| EventBridge pattern | Subset matching — only specified fields evaluated. Unspecified = ignored |
| EventBridge rule targets | Up to 5 targets per rule simultaneously |
| EventBridge event size | 256KB maximum per event |
| EventBridge at-least-once | Duplicate delivery possible — targets must be idempotent |
| Kinesis shard capacity | Write: 1 MB/s or 1,000 rec/s. Read: 2 MB/s shared (standard) |
| Kinesis partition key | Determines shard. Low cardinality = hot shard = throttling |
| Standard consumer throughput | Shared 2 MB/s per shard across ALL consumers |
| Enhanced fan-out throughput | Dedicated 2 MB/s per shard PER consumer. Push-based ~70ms |
| Kinesis retention default | 24 hours. Extended up to 365 days ($0.10/GB/month) |
| Kinesis On-Demand | Auto-scales capacity. Best for unpredictable or variable traffic |
| KCL DynamoDB requirement | KCL creates DynamoDB table for coordination — provision adequate capacity |
| Firehose min latency | 60 seconds minimum buffer interval — not true real-time |
| Firehose Lambda response | Must return ALL records with Ok/Dropped/ProcessingFailed for each |

| Topic | Key Fact |
|---|---|
| Firehose format conversion | JSON → Parquet/ORC natively via Glue schema — zero code needed |
| Firehose Redshift delivery | Delivers to S3 first then issues COPY command — not direct stream |
| MSK vs Kinesis | MSK = Kafka protocol, open source, no vendor lock-in. Kinesis = AWS-native, simpler |
| MSK min.insync.replicas | Set to 2 in production. Default 1 = single point of write failure |
| MSK partition count | Cannot decrease after topic creation — only increase |
| MSK Serverless auth | IAM only — no SASL/SCRAM or mTLS in Serverless mode |
| Idempotency strategies | Natural idempotency, DB deduplication table, DynamoDB conditional write, FIFO dedup |
| Outbox pattern | Write DB update + outbox record in same transaction → separate process sends to SQS |
| Poison pill detection | Message in DLQ after maxReceiveCount failures. Alert immediately |
| DLQ retention | Set longer than source queue (source: 4 days, DLQ: 14 days) |
| FIFO GroupId bottleneck | Single MessageGroupId = one consumer at a time — high-volume single group = bottleneck |

*Category 9: Messaging & Event-Driven — Complete Interview Guide*
*Excluded topics: CodeBuild, CodePipeline, CloudFormation*